

DOCUMENTACION TECNICA

Instalación | Control de Versiones | Historias de Usuario

Sistema Cosecha Red — v1.0

Proyecto	Cosecha Red — Plataforma Digital de Comercialización Agrícola
Categoría	Documentación Técnica
Stack tecnológico	React Node.js + FastAPI PostgreSQL
Versión	1.0.0
Fecha	19/05/2026
Elaborado por	Grupo #6

1. Guía de Instalación

1.1 Requisitos previos del sistema

Antes de instalar el proyecto, debe asegurarse de tener instalados los siguientes componentes en el sistema. Las versiones indicadas son las mínimas recomendadas.

Herramienta	Versión mínima	Propósito
Node.js	22.x LTS	Runtime del backend y herramientas de build del frontend
npm	10.x	Gestor de paquetes (incluido con Node.js)
PostgreSQL	15+	Motor de base de datos relacional
Git	2.40+	Control de versiones del código fuente
VS Code	1.85+	IDE recomendado (opcional pero sugerido)
Postman	Cualquiera	Herramienta para probar la API REST (opcional)

1.2 Estructura del proyecto

El repositorio está organizado en un monorepo con dos directorios principales: uno para el backend y otro para el frontend.

```
cosecha-en-linea/
├── backend/
│   ├── app/
│   │   ├── main.py
│   │   ├── config/
│   │   ├── database/
│   │   ├── models/
│   │   ├── schemas/
│   │   ├── routes/
│   │   ├── services/
│   │   ├── middleware/
│   │   ├── auth/
│   │   └── utils/
│   ├── requirements.txt
│   ├── .env
│   └── venv/
├── frontend/
│   ├── src/
│   │   ├── api/           # Funciones Axios para llamadas a la API
│   │   ├── components/    # Componentes React reutilizables
│   │   ├── context/       # Contextos de React (auth, usuario)
│   │   ├── hooks/         # Custom hooks
│   │   ├── pages/         # Vistas por ruta (Login, Catalogo, etc.)
│   │   ├── router/        # Configuración de rutas y rutas protegidas
│   │   └── main.jsx        # Punto de entrada de React
│   ├── .env.example
│   ├── index.html
│   └── package.json
├── database/
│   └── schema.sql         # Script SQL de creación de tablas
└── README.md
```

1.3 Pasos de instalación

Paso 1 — Clonar el repositorio

```
# Clonar el repositorio
git clone https://github.com/egvalle/Analisis1\_Proyecto.git

# Acceder al directorio del proyecto
cd Analisis1_Proyecto
```

Paso 2 — Configurar la base de datos

```
# Iniciar PostgreSQL (si no está corriendo)
sudo service postgresql start      # Linux
# o abrirlo desde pgAdmin en Windows/Mac

# Crear la base de datos
psql -U postgres -c "CREATE DATABASE CosechaRedDB;"

# Ejecutar el script de creacion de tablas
psql -U postgres -d cosecha_red -f database/schema.sql

# Verificar que las tablas se crearon correctamente
psql -U postgres -d cosecha_red -c "\dt"
```

Paso 3 — Configurar el backend

```
# Entrar al directorio backend
cd backend

# Instalar dependencias
pip uvicorn install

# Copiar la plantilla de variables de entorno
cp .env.example .env

# Editar el archivo .env con tus datos (ver sección 1.4)
code .env      # VS Code, o cualquier editor de texto
```

Paso 4 — Configurar el frontend

```
# Entrar al directorio frontend
cd frontend

# Instalar dependencias
npm install

# Copiar la plantilla de variables de entorno
cp .env.example .env

# Editar el archivo .env (ver sección 1.4)
code .env
```

Paso 5 — Ejecutar el proyecto en modo desarrollo

```
# Terminal 1: iniciar el backend
cd backend
uvicorn app.main:app --reload
# El servidor estará disponible en: http://localhost:3000
```

```
# Terminal 2: iniciar el frontend
cd frontend
npm run dev
# La aplicacion estara disponible en: http://localhost:5173
```

1.4 Variables de entorno

Backend — archivo backend/.env

```
# — Servidor —————
PORT=3000
NODE_ENV=development

# — Base de datos —————
DB_HOST=localhost
DB_PORT=5432
DB_NAME=cosecha_red
DB_USER=postgres
DB_PASSWORD=1234

# — Autenticacion JWT —————
SECRET_KEY=clave_super_segura
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=60

# — CORS —————
CORS_ORIGIN=http://localhost:5173
```

Frontend — archivo frontend/.env

```
# — API Backend —————
VITE_API_URL=http://localhost:3000/api/v1
```

1.5 Scripts disponibles

Directorio	Comando	Descripcion
backend	npm run dev	Inicia el servidor con nodemon (recarga automatica)
backend	npm start	Inicia el servidor en modo produccion
backend	npm test	Ejecuta las pruebas unitarias con Jest
backend	npm run lint	Ejecuta ESLint para revision de codigo
frontend	npm run dev	Inicia el servidor de desarrollo de Vite
frontend	npm run build	Genera el build de produccion en /dist
frontend	npm run preview	Previsualiza el build de produccion localmente
frontend	npm run lint	Ejecuta ESLint en el codigo del frontend

1.6 Guía de configuración del entorno de desarrollo

IDE recomendado: Visual Studio Code

Se recomienda Visual Studio Code con las siguientes extensiones instaladas para mantener consistencia en el equipo:

- ESLint — Detección de errores y malas prácticas en JavaScript
- Prettier — Formateo automático de código
- Tailwind CSS IntelliSense — Autocompletado de clases de Tailwind
- REST Client o Thunder Client — Pruebas de API directamente en VS Code
- GitLens — Visualización avanzada del historial de Git
- PostgreSQL (cweijan) — Gestión de la base de datos desde el IDE

Configuración de ESLint y Prettier

El proyecto incluye archivos de configuración (.eslintrc.json y .prettierrc) en cada directorio. Al instalar las dependencias con npm install estos se aplican automáticamente. Se recomienda activar "Format on Save" en VS Code para que Prettier formatee el código al guardar.

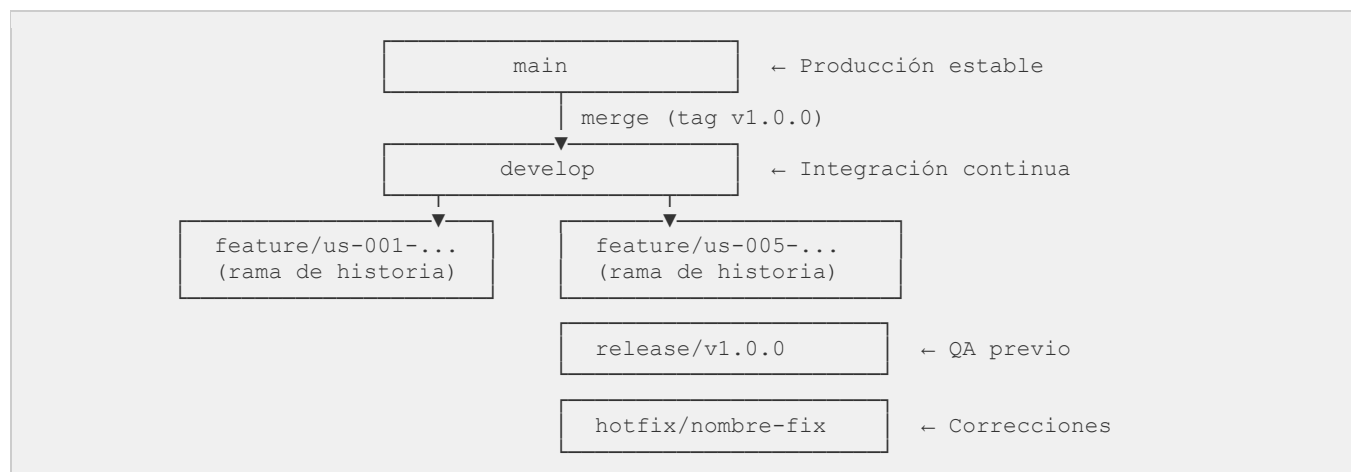
Convenciones de código

- Nombrado de archivos: camelCase para archivos JS/JSSX, kebab-case para CSS.
- Nombrado de variables y funciones: camelCase.
- Nombrado de componentes React: PascalCase.
- Nombrado de constantes globales: UPPER_SNAKE_CASE.
- Comentarios: en español para lógica de negocio, en inglés para código técnico genérico.
- Longitud máxima de línea: 100 caracteres.

2. Control de Versiones

2.1 Estrategia de ramas — GitHub

El proyecto adopta GitHub como estrategia de control de versiones. Esta convención establece un flujo claro entre ramas de desarrollo, funcionalidades, preparación de releases y correcciones urgentes.



2.2 Descripción de ramas

main	Rama de producción. Solo recibe merges desde release/* o hotfix/*. Cada merge genera un tag de versión.
-------------	---

develop	Rama de integración. Recibe las ramas feature/* completadas. Representa el estado actual del desarrollo.
feature/*	Una rama por historia de usuario o funcionalidad. Se crea desde develop y se fusiona de vuelta a develop via pull request.
release/*	Se crea desde develop cuando se prepara un release. Permite correcciones menores de QA antes de fusionar a main.
hotfix/*	Correcciones urgentes en producción. Se crean desde main y se fusionan tanto a main como a develop.

2.3 Convención de nombres de ramas

```
# Historias de usuario
feature/us-001-registro-usuario
feature/us-004-publicar-producto
feature/us-007-contactar-productor

# Releases
release/v1.0.0
release/v1.1.0

# Correcciones urgentes
hotfix/fix-login-token-expiry
hotfix/fix-catalogo-filtro-precio
```

2.4 Convención de mensajes de commit

Los mensajes de commit siguen el estandar Conventional Commits para mantener un historial claro y facilitar la generación automática de changelogs.

```
# Formato
tipo(alcance): descripción breve en presente

# Tipos validos
feat      → Nueva funcionalidad
fix       → Corrección de error
docs      → Cambios en documentación
style     → Formato, sin cambios lógicos
refactor  → Refactorización sin nueva funcionalidad
test      → Agregar o corregir pruebas
chore     → Tareas de mantenimiento (dependencias, config)

# Ejemplos
feat(auth): agregar endpoint de registro de usuario
feat(publicaciones): implementar filtro por ubicación
fix(interacciones): corregir unicidad de contacto duplicado
docs(api): agregar ejemplos de request en sección 3.8
test(auth): agregar prueba unitaria para login fallido
```

2.5 Flujo de trabajo de una historia de usuario

El siguiente flujo describe el ciclo de vida completo de una rama de funcionalidad, desde su creación hasta su integración en develop:

1. Paso 1: Crear la rama desde develop: `git checkout -b feature/us-XXX-nombre develop`
2. Paso 2: Desarrollar la funcionalidad con commits atomicos siguiendo la convención.
3. Paso 3: Ejecutar las pruebas locales antes de subir: `npm test`

4. Paso 4: Subir la rama al repositorio remoto: `git push origin feature/us-XXX-nombre`
5. Paso 5: Abrir un Pull Request hacia develop con descripción de los cambios realizados.
6. Paso 6: Otro miembro del equipo revisa el código (code review).
7. Paso 7: Si hay observaciones, se corrigen en la misma rama con nuevos commits.
8. Paso 8: Una vez aprobado, se fusiona a develop y se elimina la rama de feature.

2.6 Tags y releases

Cada versión estable del sistema se etiqueta en la rama main con un tag semántico siguiendo el formato vMAJOR.MINOR.PATCH (SemVer):

MAJOR (v1.x.x → v2.x.x)	Cambios que rompen compatibilidad con versiones anteriores.
MINOR (v1.0.x → v1.1.x)	Nuevas funcionalidades que mantienen compatibilidad.
PATCH (v1.0.0 → v1.0.1)	Correcciones de errores sin nuevas funcionalidades.

```
# Crear tag del primer release
git tag -a v1.0.0 -m "Release v1.0.0 - Primera versión del sistema Cosecha Red"

# Subir el tag al repositorio remoto
git push origin v1.0.0

# Ver todos los tags
git tag --list
```

3. Historias de Usuario

3.1 Introducción

Las historias de usuario describen las funcionalidades del sistema desde la perspectiva de cada tipo de usuario. Cada historia sigue el formato: Como [rol], necesito [accion], para [beneficio]. Los criterios de aceptación definen las condiciones que deben cumplirse para considerar la historia completada.

Total de historias	13 historias de usuario
Roles	Productor, Comprador, Administrador
Formato	Como / Necesito / Para + Criterios de aceptación
Prioridades	Alta, Media, Baja
Trazabilidad	Cada historia referencia el CU correspondiente

3.2 Historias de usuario — Acceso

US-001 — Registro de cuenta	
Como...	usuario nuevo (productor o comprador)
Necesito...	poder crear una cuenta en la plataforma indicando mis datos personales y mi rol
Para...	acceder a las funcionalidades del sistema según el rol que me corresponde

Criterios de aceptación	• El formulario solicita nombre completo, correo, teléfono, contraseña y rol (Productor / Comprador o Ambos). • El sistema valida que el correo no este previamente registrado. • La contraseña debe tener mínimo 8 caracteres. • El rol Administrador no aparece como opción en el registro público. • Al crear la cuenta, el usuario queda autenticado automáticamente. • El sistema redirige al usuario a la pantalla principal según su rol.
Prioridad	Alta
Caso de uso relacionado	CU-ACC-002 — Crear Cuenta

US-002 — Inicio de sesion	
Como...	usuario registrado
Necesito...	iniciar sesion con mi correo y contraseña
Para...	acceder a las funcionalidades de la plataforma que corresponden a mi rol
Criterios de aceptación	• El sistema valida las credenciales y genera un token JWT al iniciar sesion. • Si las credenciales son incorrectas se muestra un mensaje genérico sin especificar que campo fallo. • Si la cuenta esta inactiva se informa al usuario y no se genera sesion. • El usuario es redirigido a la pantalla principal correspondiente a su rol.
Prioridad	Alta
Caso de uso relacionado	CU-ACC-001 — Iniciar Sesion

3.3 Historias de usuario — Perfil

US-003 — Gestión de perfil	
Como...	usuario autenticado (cualquier rol)
Necesito...	poder consultar y actualizar mi nombre, teléfono y contraseña
Para...	mantener mis datos de contacto correctos para que los compradores puedan comunicarse conmigo
Criterios de aceptación	• El perfil muestra nombre, correo (solo lectura), teléfono, rol y reputación pública. • El usuario puede editar el nombre y el teléfono. • El cambio de contraseña requiere ingresar la contraseña actual. • Si el productor actualiza su teléfono, todas sus publicaciones activas lo reflejan automáticamente. • El correo electrónico y el rol no son editables desde el perfil.
Prioridad	Media

Caso de uso relacionado	CU-PERF-001 — Gestionar Perfil de Usuario
--------------------------------	---

3.4 Historias de usuario — Productor

US-004 — Publicar producto	
Como...	productor
Necesito...	poder publicar mis productos en el catálogo indicando nombre, cantidad, unidad, precio y lugar de entrega
Para...	que los compradores puedan encontrar mis productos y contactarme para negociar
Criterios de aceptación	• El formulario incluye: nombre del producto, cantidad, unidad de medida, precio de referencia, ubicación de entrega y descripción opcional. • Los datos de contacto del productor se toman automáticamente de su perfil. • El sistema valida que los campos obligatorios estén completos y que cantidad y precio sean mayores a cero. • La publicación aparece en el catálogo en tiempo real tras ser creada. • El precio de referencia no puede modificarse una vez publicado.
Prioridad	Alta
Caso de uso relacionado	CU-PROD-001 — Publicar Producto

US-005 — Editar publicación	
Como...	productor
Necesito...	poder editar los datos de una publicación que ya cree
Para...	mantener actualizada la información de mis productos cuando la disponibilidad o las condiciones cambian
Criterios de aceptación	• El formulario de edición precarga los valores actuales de la publicación. • El productor puede modificar cantidad, unidad, ubicación y descripción. • Los cambios se reflejan en el catálogo en tiempo real. • El sistema registra la fecha de la última modificación.
Prioridad	Alta
Caso de uso relacionado	CU-PROD-002 — Editar Publicación

US-006 — Eliminar publicación	
Como...	productor
Necesito...	poder eliminar una publicación que ya no deseo mantener disponible

Para...	retirar del catálogo productos que ya no tengo disponibles para evitar que compradores intenten contactarme por ellos
Criterios de aceptación	• El sistema solicita confirmación explícita indicando que la acción es irreversible. • La publicación desaparece del catálogo en tiempo real tras la eliminación. • El historial de interacciones asociado a la publicación se conserva en el sistema.
Prioridad	Media
Caso de uso relacionado	CU-PROD-003 — Eliminar Publicación

3.5 Historias de usuario — Comprador

US-007 — Consultar catalogo	
Como...	comprador
Necesito...	poder explorar el catálogo de productos disponibles y aplicar filtros por tipo, precio y ubicación
Para...	encontrar rápidamente los productos que me interesan sin tener que revisar todo el listado
Criterios de aceptación	• El catálogo muestra todas las publicaciones activas con nombre, cantidad, precio y ubicación. • El comprador puede filtrar por tipo de producto, rango de precio y ubicación de entrega. • Si no hay resultados con los filtros aplicados, el sistema lo indica y ofrece limpiarlos. • El número de teléfono del productor no es visible en el catálogo ni en el detalle de la publicación. • El detalle de cada publicación muestra la reputación del productor.
Prioridad	Alta
Caso de uso relacionado	CU-COMP-001 — Consultar Catálogo de Productos

US-008 — Contactar productor	
Como...	comprador
Necesito...	poder ver el número de teléfono de un productor cuando decido contactarlo para negociar una compra
Para...	poder llamar o enviar un mensaje al productor directamente para acordar precio, cantidad y lugar de entrega
Criterios de aceptación	• Al presionar "Contactar al productor" el sistema registra automáticamente la interacción.

	• El número de teléfono del productor se muestra solo en ese momento. • No se generan registros duplicados si el comprador contacta al mismo productor por la misma publicación más de una vez. • Si la publicación fue eliminada antes del contacto, el sistema lo indica sin generar interacción.
Prioridad	Alta
Caso de uso relacionado	CU-COMP-002 — Contactar Productor

3.6 Historias de usuario — Historial

US-009 — Consultar historial	
Como...	productor o comprador
Necesito...	poder consultar mi historial de interacciones registradas en la plataforma
Para...	tener trazabilidad de con quien he interactuado, cuando y si ya los he calificado
Criterios de aceptación	• El historial muestra interacciones ordenadas por fecha descendente. • Cada registro indica el nombre del otro usuario, el producto involucrado, la fecha y el estado de valoración. • El historial es de solo lectura; el usuario no puede modificarlo. • El historial se conserva, aunque la publicación asociada sea eliminada posteriormente.
Prioridad	Media
Caso de uso relacionado	CU-HIST-001 — Consultar Historial de Interacciones

3.7 Historias de usuario — Valoraciones

US-010 — Calificar usuario	
Como...	productor o comprador
Necesito...	poder calificar de 1 a 5 estrellas a un usuario con quien tuve contacto registrado
Para...	contribuir al sistema de reputación para que otros usuarios puedan tomar decisiones informadas
Criterios de aceptación	• Solo se puede calificar si existe una interacción previa registrada entre ambos usuarios. • Cada usuario solo puede enviar una valoración por interacción. • La valoración requiere una puntuación de 1 a 5 estrellas; el comentario es opcional. • La valoración queda en estado "Pendiente" hasta que el administrador la apruebe. • El usuario recibe confirmación de que su valoración será visible tras la revisión.

Prioridad	Media
Caso de uso relacionado	CU-VAL-001 — Calificar Usuario

US-011 — Ver reputación de usuario	
Como...	comprador o productor
Necesito...	poder ver la reputación publica de otro usuario de la plataforma
Para...	tomar decisiones informadas sobre con quien interactuar basándome en la experiencia de otros usuarios
Criterios de aceptación	• La reputación muestra el promedio de estrellas, el número de valoraciones aprobadas y los comentarios. • Solo las valoraciones aprobadas por el administrador son visibles e influyen en el promedio. • La reputación del productor es visible desde el detalle de sus publicaciones en el catálogo. • Si el usuario no tiene valoraciones aprobadas, el sistema lo indica claramente.
Prioridad	Media
Caso de uso relacionado	CU-VAL-002 — Visualizar Reputación de Usuario

3.8 Historias de usuario — Administrador

US-012 — Gestionar usuarios y validar valoraciones	
Como...	administrador
Necesito...	poder gestionar las cuentas de usuarios y revisar las valoraciones enviadas antes de publicarlas
Para...	garantizar que solo participen usuarios activos y confiables, y que el sistema de reputación sea integro
Criterios de aceptación	• El panel de administración muestra métricas clave: usuarios, valoraciones pendientes y publicaciones activas. • El administrador puede buscar, filtrar, activar y desactivar cuentas de usuarios. • El administrador puede aprobar o rechazar valoraciones pendientes. • Al rechazar una valoración debe ingresar obligatoriamente el motivo del rechazo. • Las valoraciones aprobadas se hacen públicas y actualizan el promedio de reputación en tiempo real. • Todos los cambios quedan registrados en el historial de auditoría.
Prioridad	Alta

Caso de uso relacionado	CU-ADM-001, CU-ADM-002, CU-ADM-003
--------------------------------	------------------------------------

US-013 — Crear cuenta de administrador	
Como...	administrador existente
Necesito...	poder crear nuevas cuentas con rol Administrador desde el panel de administración
Para...	incorporar nuevos administradores de forma controlada sin exponerlo en el registro publico
Criterios de aceptación	• La opción de crear cuenta de administrador solo está disponible dentro del panel de administración. • El rol Administrador no aparece como opción en el formulario de registro público. • El formulario solicita nombre, correo y contraseña temporal. • La acción queda registrada en el historial de auditoria con fecha, hora y autor.
Prioridad	Alta
Caso de uso relacionado	CU-ADM-004 — Crear Cuenta de Administrador